Q1. What is the `sizeof` Operator and Its Importance? (5 Marks)

Definition

The `sizeof` operator in C is a unary operator used to determine the memory size of a variable, data type, or expression in bytes. It returns a value of type `size_t`.

Syntax

```
sizeof(data_type);
sizeof(variable);
```

Example

```
#include <stdio.h>

int main() {
    int a = 10;
    float b = 5.5;

    printf("Size of int = %zu bytes\n", sizeof(a));
    printf("Size of float = %zu bytes\n", sizeof(b));

    return 0;
}
```

Output

```
Size of int = 4 bytes
Size of float = 4 bytes
```

Note: Sizes may vary depending on the system.

Importance of `sizeof` Operator

1. Memory Management: Helps determine the amount of memory occupied by variables and data types.
2. Dynamic Memory Allocation: Used with functions such as `malloc()` and `calloc()` to allocate the required memory.
3. Portability: Makes programs adaptable to different computer architectures.
4. Array Size Calculation: Helps calculate the number of elements in a fixed-size array.
5. Memory Optimization: Helps programmers understand and manage memory usage efficiently.

Conclusion

The `sizeof` operator is an important feature of C programming that helps in memory management, dynamic memory allocation, and writing efficient and portable programs

Q2. Compare Static vs Dynamic Memory Allocation with Example. (10 Marks)

Definition of Static Memory Allocation

Static memory allocation refers to allocating memory with a fixed size determined before program execution. The size of the allocated memory cannot be changed during execution.

Example:

```
int arr[5] = {10, 20, 30, 40, 50};
```

Definition of Dynamic Memory Allocation

Dynamic memory allocation refers to allocating memory during program execution according to the program's requirements. It uses functions such as `malloc()`, `calloc()`, `realloc()`, and `free()`.

Example:

```
int *arr = malloc(5 * sizeof(int));
```

Differences Between Static and Dynamic Memory Allocation

Static Memory Allocation	Dynamic Memory Allocation
Memory size is fixed before execution.	Memory size is decided during execution.
Uses fixed-size declarations.	Uses functions like `malloc()` and `calloc()`.
Memory size cannot be resized.	Memory can be resized using `realloc()`.
Generally faster and simpler.	Requires additional memory management.
Fixed-size local arrays are usually stored on the stack.	Dynamically allocated memory is stored on the heap.
Local automatic memory is released automatically when its lifetime ends.	Allocated memory must be released using `free()`.
May waste memory if the declared size is too large.	Memory can be allocated based on actual requirements.
Suitable for fixed-size data structures.	Suitable for variable-size data structures.

C Program for Static Memory Allocation

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output:

```
10 20 30 40 50
```

C Program for Dynamic Memory Allocation

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5;
    int *arr = malloc(n * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        arr[i] = (i + 1) * 10;
        printf("%d ", arr[i]);
    }

    free(arr);
    return 0;
}
```

Output:

```
10 20 30 40 50
```

Advantages of Static Memory Allocation

1. Simple to implement and manage.
2. No explicit memory allocation or deallocation is required.
3. Suitable for fixed-size arrays and data structures.

Advantages of Dynamic Memory Allocation

1. Provides flexibility in memory usage.
2. Allows memory allocation during runtime.

3. Supports dynamic data structures such as linked lists, stacks, queues, and trees.

Conclusion

Static memory allocation is suitable when the required memory size is known in advance, whereas dynamic memory allocation is useful when memory requirements are determined or change during program execution

Q3. Solve Recursion-Based String Reversal. (5 Marks)

Definition

Recursion is a programming technique in which a function calls itself to solve a problem.

String reversal using recursion involves reversing a string by recursively processing each character and printing it in reverse order.

Algorithm

1. Start the program.
2. Read a string from the user.
3. Call the recursive function `reverse()`.
4. Check whether the current character is the null character (`'\0'`).
5. If not, recursively call the function for the next character.
6. Print the characters while returning from recursive calls.
7. Stop the program.

C Program

```
#include <stdio.h>
#include <string.h>

void reverse(char str[], int index) {
    if (str[index] == '\0')
        return;

    reverse(str, index + 1);
    printf("%c", str[index]);
}

int main() {
    char str[100];

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    str[strcspn(str, "\n")] = '\0';

    printf("Reversed string: ");
    reverse(str, 0);

    return 0;
}
```

Output

```
Enter a string: HELLO
Reversed string: OLLEH
```

Working Explanation

Consider the string `CAT`.

Recursive Function Calls:

```
reverse("CAT", 0)
    reverse("CAT", 1)
        reverse("CAT", 2)
            reverse("CAT", 3)
                return
            print T
        print A
    print C
```

Final Output: `TAC`

Time and Space Complexity

- Time Complexity: (O(n)), because each character is processed once.
- Space Complexity: (O(n)), due to the recursive function call stack.

Conclusion

String reversal using recursion is achieved by calling the function until the end of the string is reached and printing characters in reverse order while returning from the recursive calls.